Electronics & Computing Club

Digital Dice Project

Build & Code: Arduino Workshop

Written by Charles Ross on December 29th, 2025

Section 1
Overview

Have you ever rolled dice around and thought, *"man, the lack of clicky pushbuttons isn't satisfying enough for me, and I wish I could automate the rolling!"*? Well, you're in luck! This project will be sure to satisfy all your strange dice-related desires.

With this project, you can make a button-activated dice with a 7-segment digital display. It's going to be a little more unwieldy than a regular die, sure, but working on this project will help you familiarize yourself with randomness in coding, and you'll learn about the hardware that goes into displaying numbers.

You'll start by randomly generating a number whenever you click a button. Then, you will send this number out to a circuit that will display to your 7-segment display - all in all, pretty simple. As you wire the circuit, the challenge is to understand *how* exactly you take your randomized number and send it out to your display.

The key to figuring this out is to understand that a 7-segment display is really just a bunch of LED bars arranged in an 8 pattern. You will need a circuit that knows when to light up the right LEDs to form the number you want. The circuit we need to perform this task is what we call a *decoder*.

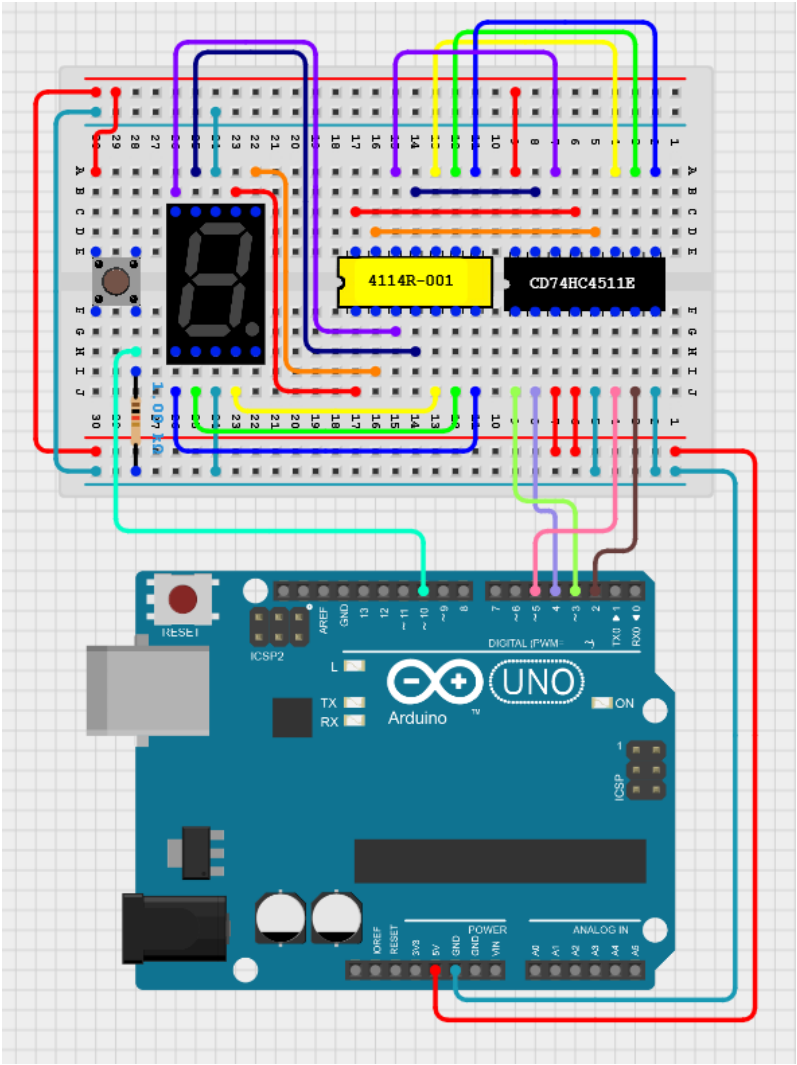
Section 2
Required Components

Here's a list of what you'll need:

- 1x 7-segment display
- 1x Momentary Switch
- 1x 1kΩ Resistor
- 1x DIP Resistor
- 1x BCD to 7-segment Display Decoder (CD74HC4511E)
- 1x Breadboard
- 1x Arduino Uno
- Assorted Breadboard Wires
- Assorted Jumper Wires

Section 3
Circuit Assembly

Use this diagram as reference:



https://app.cirkitdesigner.com/project/4e8fcede-d3e9-4134-b170-d8b8872891a1

Please refer to the image above for the complete circuit wiring. A link to the online schematic can be found under the image in case you need higher resolution.

If this is your first time using a breadboard, please refer to our *Breadboard Guide*.

1. **Start by connecting your 2 power bus lines together**
   The two + lines should be connected with a wire and vice-versa for the two - lines.

2. **Place your components as indicated by the diagram**
   For all components other than the resistor, make sure they're in the middle of the breadboard, where the little divot is placed. The resistor should have one side connected to one of the button pins, with the other side connected to the - line.

3. **Start by wiring the power connections**
   Connect the Arduino 5V pin to the + line and one of the Arduino GND pins to the - line. For the button, connect the pin opposite to where the resistor is to the + line. Connect the middle pins on both sides of the 7-segment display to the - line. Connect the pins on the decoder (black chip) to the + and – lines as indicated in the diagram (if you'd like some assistance, be sure to ask a volunteer).

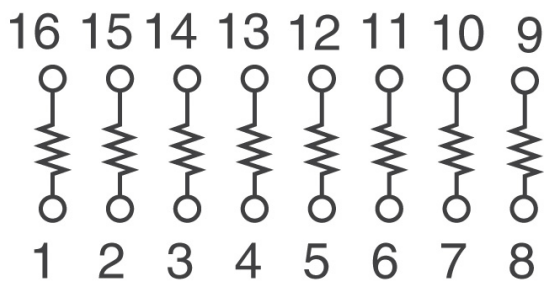
4. **Wire up your signal connections to the Arduino**
   Connect the Arduino pin 10 to the resistor-button connection. Then, connect Arduino pins 2,3,4 and 5 to the decoder as shown in the diagram.

5. **Wire up your connections between the DIP resistor and the decoder**
   Use the diagram as reference. It's important to understand that the resistor pack doesn't require connections to be made in any order, so you can customize your connections to make it less cluttered if you'd like. That said, what is important is that you wire the connections in the same way on the other side. To make it easier to track, the wires from both sides have the same color, so you can identify which decoder pin routes to which pin of the 7-segment display.

6. **Wire up your connections between the resistor pack and the 7-segment display.** If you have changed the wiring from the diagram, remember to make sure they are correctly wired. Otherwise, refer to the diagram connections.

What you're wiring up here is the display circuit and input button for your dice. The decoder (black chip) is a digital circuit that will take a binary number sent by your Arduino and send out a 1 or a 0 to each pin of the 7-segment display depending on the number it receives. The idea is that each output pin of the decoder is associated with a bar on the 7-segment display, and it will activate or deactivate the bar based on the number you give it.

The resistor pack simply contains seven vertically oriented resistors. They are just here to limit the current provided to the LEDs, so they don't burn out. The inside of the component looks like this (with pin numbers shown):

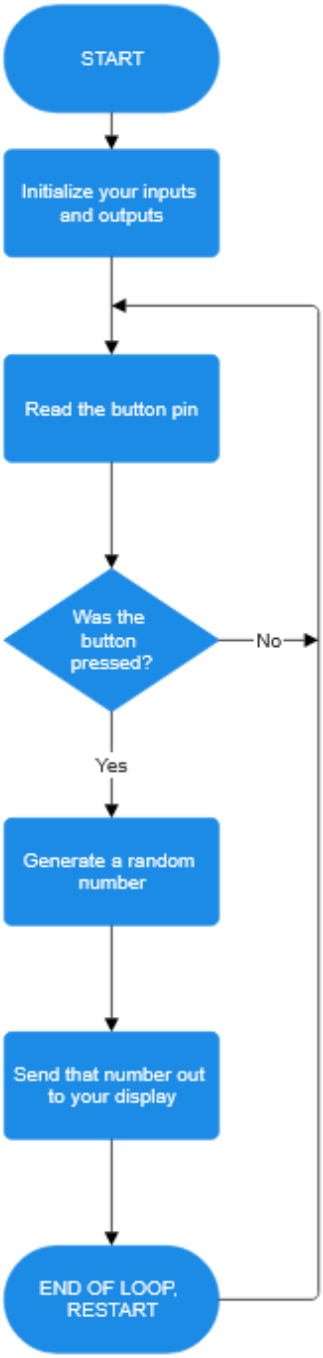


https://www.chipfind.net/static/photo/resistors/68585.jpg

Finally, the button has another resistor, this one acting as a *pull-down* resistor. This is to avoid what we know as *floating inputs*, which are inputs that can fluctuate to a 1 (digital HIGH) or 0 (digital LOW), usually caused by static electricity or other electromagnetic interference. The *pull-down* resistor ensures your input is entirely dependent on the button press.

Section 4
Coding

Use this flowchart to guide your coding:



The code will start with initializing your inputs and outputs. What this means is you tell the Arduino whether the pin being used is an input or an output. You can perform this with the *pinMode()* function, whose syntax is as such:

```
pinMode([pin number], {OUTPUT/INPUT})
```

e.g. `pinMode(10,INPUT);`

After this, you enter an infinite loop where you continually check the button to see if it has been pressed. You do this with the *digitalRead()* function, whose syntax is as such:

```
digitalRead([pin number]);
```

e.g. `buttonState = digitalRead(10);`

Once you get the button state, you need to check if it is 1 or 0. If it's 1, the button was physically pressed. You need to make the program decide when it will roll the dice based on the result of this button state.

Once this decision has been made, you can generate a random number using the *random()* function. This is how you use the random function to create a number randomized from 1-6:

```
randomNumber = random(1,7);
```

Finally, you need to output the randomized number to the pins 2, 3, 4, and 5. The function you use to output a bit to a pin is this *digitalWrite()*, which is written as such:

```
digitalWrite([pin number],{HIGH/LOW})
```

The biggest challenge here is that *digitalWrite()* only changes one pin at a time, while a number like 6 needs 3 pins (because 6 is equivalent to 101 in binary). This means that you need to do some strange bit-related manipulations to output the number to your 7-segment display.

To avoid the trouble, you are provided with a custom-made function that can perform the task described. Just copy-paste the bit of code on the next page at the bottom of your program code, and use the function within your program as though you would any other function. You are welcome to create your own based on what you've learned, as well. If you have any questions, feel free to ask a volunteer.

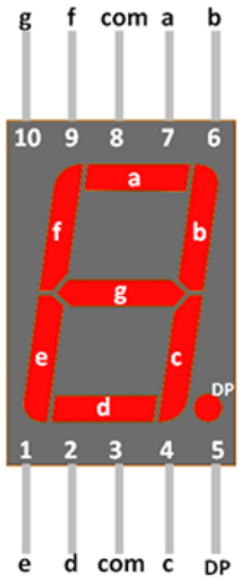
```
void outputToPins(int numberOutput){
  for(int i = 0; i < 4; i++){
    digitalWrite(ledPins[i],(bool)(numberOutput & 1));
    numberOutput = numberOutput >> 1;
  }
}
```

Make sure to refer to the *Coding Dictionary* for useful functions, libraries and terms. They may help you accomplish this task.

Section 5
Challenge Yourself

1. Decoder within the program

Remove the decoder IC component and perform the decoding within your program instead. This can be done through the use of lookup tables, which will take a number and associate it with a different number (or in our case, a series of bits). These bits will represent each LED that you output. You can use this image to help you wire the 7-segment display to your Arduino, and to create your bit series.



https://components101.com/displays/7-segment-display-pinout-working-datasheet

Here's an example of what a lookup table can look like. In this example, the 0 is represented by 0b1111110 because the first 6 bits (a-f) should light up and only the bit for g should be turned off.

Your input to the lookup table is the index (refer to the array in the *Coding Dictionary*), and the output is the number that corresponds to the index.

```
const int lookup_table[10] =
{0b1111110,0b0110000,0b1101101,0b1111001,0b0110011,0b1011011,0b1011111,0b1110000,
0b1111111,0b1111011};
```

2. Rolling Animation

Make a small animation before you show the generated number. You can reuse the random function to flash a bunch of different numbers before you get your result. You should also use the delay() function to prevent the animation from going too quickly.